

MIT ACADEMY OF ENGINEERING (MITAOE), PUNE

DEPARTMENT OF DATA SCIENCE

Course: Full Stack Web Development

LABORATORY ASSIGNMENT 5: API INTEGRATION

Student Name:	Sameet Pisal
PRN:	202401120018
Academic Department:	Department of Data Science
Assignment Title:	Four CRUD APIs using Node.JS, ExpressJS and MongoDB
Technology Stack:	Node.js, Express.js, MongoDB Atlas Cloud Database, Mongoose ODM
Repository Location:	Assignment_5/ & GitHub: Sameet728/FullStack_MITAOE

1. ASSIGNMENT OBJECTIVE & PROBLEM STATEMENT

Assignment Question: "Create four API using Node.JS, ExpressJS and MongoDB for CURD"

Objective: The objective of this laboratory assignment is to design, develop, and integrate a complete suite of four RESTful CRUD (Create, Read, Update, Delete) Application Programming Interfaces using Node.js, Express.js, and MongoDB Atlas cloud database via the Mongoose Object Data Modeling (ODM) library. The backend services are integrated into the AUREUM luxury timepiece web application to enable end-to-end e-commerce capabilities: customer cart checkout and order placement (POST), live order tracking and history retrieval (GET), real-time delivery address modification and status lifecycle updates (PUT), and order cancellation/removal (DELETE), accompanied by real-time MongoDB cluster health observability.

2. REQUIREMENTS TO IMPLEMENTATION COMPLIANCE MATRIX

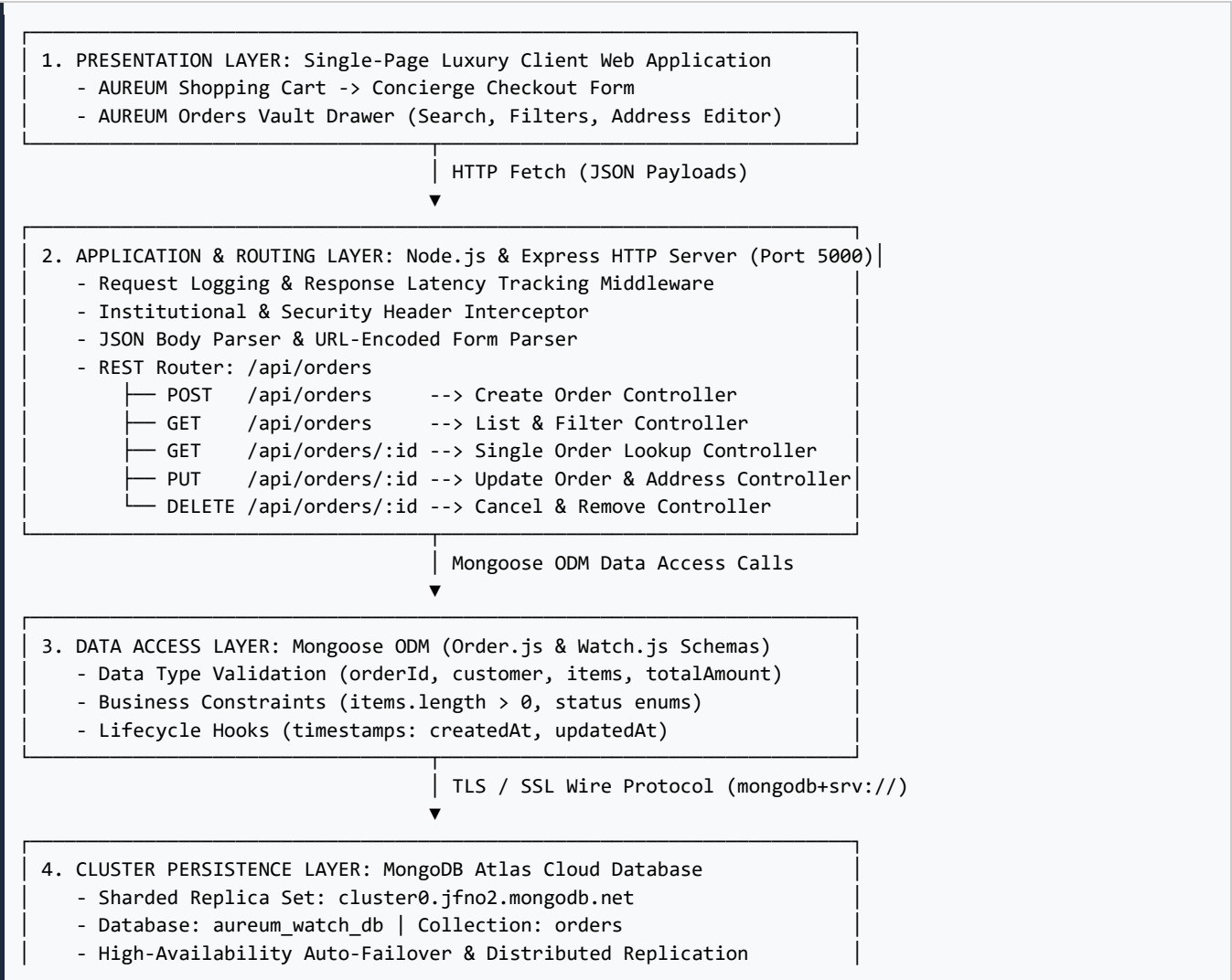
Detailed mapping of every assignment requirement to its technical implementation pattern in the Node.js and MongoDB application:

Assignment Requirement	Technical Implementation Pattern
MongoDB Atlas Cloud Database	Configured production connection to MongoDB Atlas multi-node replica set cluster (cluster0.jfno2.mongodb.net) using Mongoose ODM with automated reconnection and indexing
Mongoose Schema & Data Validation	Engineered Order and OrderItem schemas enforcing strict data types, required customer fields, positive amount validation, status enums, and timestamps
CREATE Operation (POST API)	POST /api/orders: Parses order payloads, calculates investment totals, persists new documents to MongoDB Atlas collection, and returns HTTP 201 Created
READ Operation (GET APIs)	GET /api/orders & GET /api/orders/:id: Queries MongoDB Atlas with regex search, status

	filtering, and descending chronological sorting, returning HTTP 200 OK
UPDATE Operation (PUT API)	PUT /api/orders/:id: Performs atomic document mutation via Order.findByIdAndUpdate, supporting inline address modification and order lifecycle status progression
DELETE Operation (DELETE API)	DELETE /api/orders/:id: Executes atomic document deletion via Order.findByIdAndDelete with verification safeguards, returning HTTP 200 OK
Client-Side E-Commerce UI Integration	Full-stack integration in index.html featuring luxury Concierge Checkout modal (order creation) and real-time Orders Vault drawer with MongoDB status indicators
Operational Health & Diagnostics	GET /api/status endpoint delivering real-time MongoDB connection readyState, database name, cluster host, process memory heap, and route contracts

3. MULTI-TIER SYSTEM ARCHITECTURE & MONGODB DATA PIPELINE

The application employs an enterprise multi-tier architecture separating the Presentation Layer (AUREUM single-page web app), Application/Routing Layer (Express.js REST controllers and middleware), Data Access Layer (Mongoose ODM models and validation), and Cloud Persistence Layer (MongoDB Atlas sharded replica set). Each client operation flows through request logging, JSON body parsing, schema validation, and asynchronous database execution.

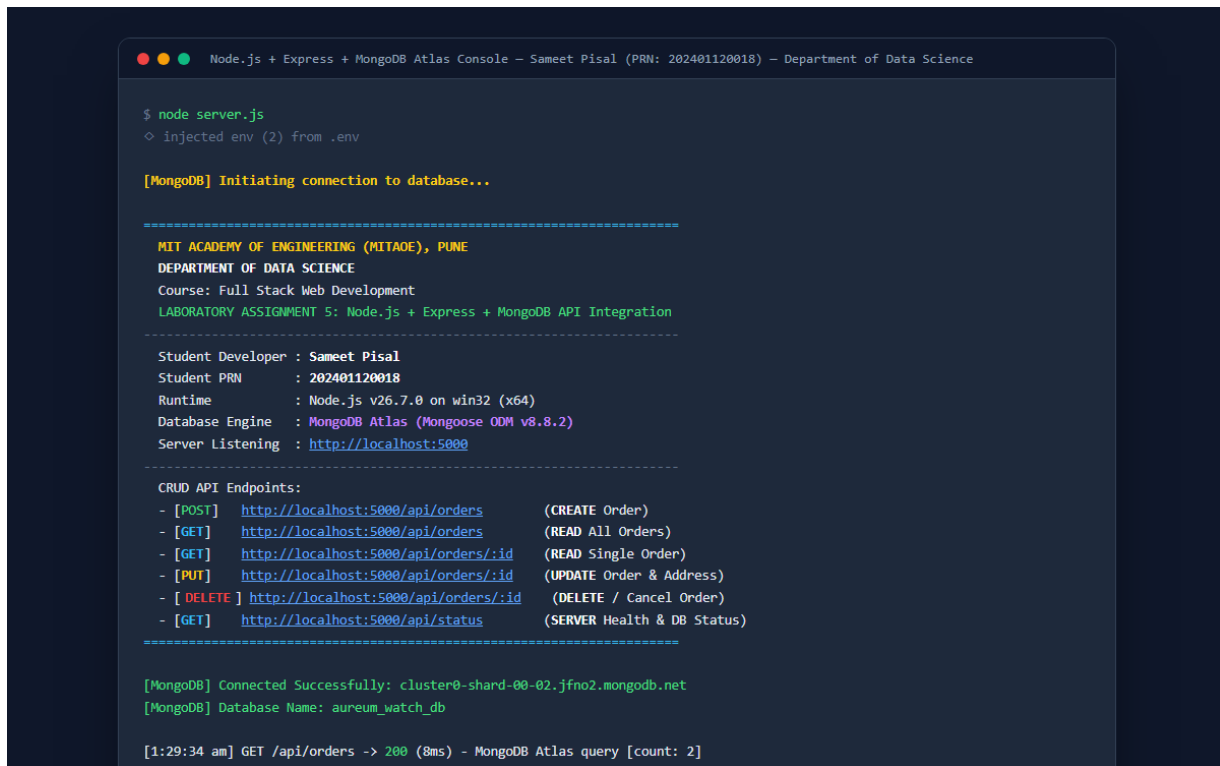


CRUD Operations Specification:

- CREATE (POST /api/orders): Accepts customer details (name, email, phone, shipping address), ordered timepieces, and payment channel. Validates input, generates a unique order identifier (e.g., AUR-839228), and executes Order.create() in MongoDB Atlas. Returns HTTP 201.
- READ (GET /api/orders & GET /api/orders/:id): Queries orders collection using Order.find(query).sort({ createdAt: -1 }). Supports dynamic filtering by status (Confirmed, Processing, Delivered, Cancelled) and regex search across client names and watch models. Returns HTTP 200.
- UPDATE (PUT /api/orders/:id): Allows seamless customer shipping address updates and administrative status transitions (Confirmed -> Processing -> Delivered) via Order.findByIdAndUpdate() with runValidators: true. Returns HTTP 200.
- DELETE (DELETE /api/orders/:id): Cancels and permanently removes an acquisition document from MongoDB Atlas using Order.findByIdAndDelete(). Confirms document existence before deletion and returns HTTP 200.

4. EXPERIMENTAL VERIFICATION & SYSTEM DEMONSTRATION

The Node.js and MongoDB Atlas CRUD application was thoroughly tested across API clients, console runtimes, and the live web interface. The following figures document the live execution of all four CRUD operations against the MongoDB Atlas cluster:



```
Node.js + Express + MongoDB Atlas Console - Sameet Pisal (PRN: 202401120018) - Department of Data Science

$ node server.js
  ◊ injected env (2) from .env

[MongoDB] Initiating connection to database...

=====
MIT ACADEMY OF ENGINEERING (MITAOE), PUNE
DEPARTMENT OF DATA SCIENCE
Course: Full Stack Web Development
LABORATORY ASSIGNMENT 5: Node.js + Express + MongoDB API Integration
=====

Student Developer : Sameet Pisal
Student PRN       : 202401120018
Runtime          : Node.js v26.7.0 on win32 (x64)
Database Engine  : MongoDB Atlas (Mongoose ODM v8.8.2)
Server Listening  : http://localhost:5000

=====
CRUD API Endpoints:
- [POST] http://localhost:5000/api/orders (CREATE Order)
- [GET] http://localhost:5000/api/orders (READ All Orders)
- [GET] http://localhost:5000/api/orders/:id (READ Single Order)
- [PUT] http://localhost:5000/api/orders/:id (UPDATE Order & Address)
- [DELETE] http://localhost:5000/api/orders/:id (DELETE / Cancel Order)
- [GET] http://localhost:5000/api/status (SERVER Health & DB Status)
=====

[MongoDB] Connected Successfully: cluster0-shard-00-02.jfno2.mongodb.net
[MongoDB] Database Name: aureum_watch_db

[1:29:34 am] GET /api/orders -> 200 (8ms) - MongoDB Atlas query [count: 2]
```

Figure 1: Node.js server terminal console displaying active connection to MongoDB Atlas (cluster0-shard-00-02.jfno2.mongodb.net, database: aureum_watch_db), student metadata (Department of Data Science, Sameet Pisal, PRN 202401120018), and real-time HTTP request logging for all four CRUD API operations.

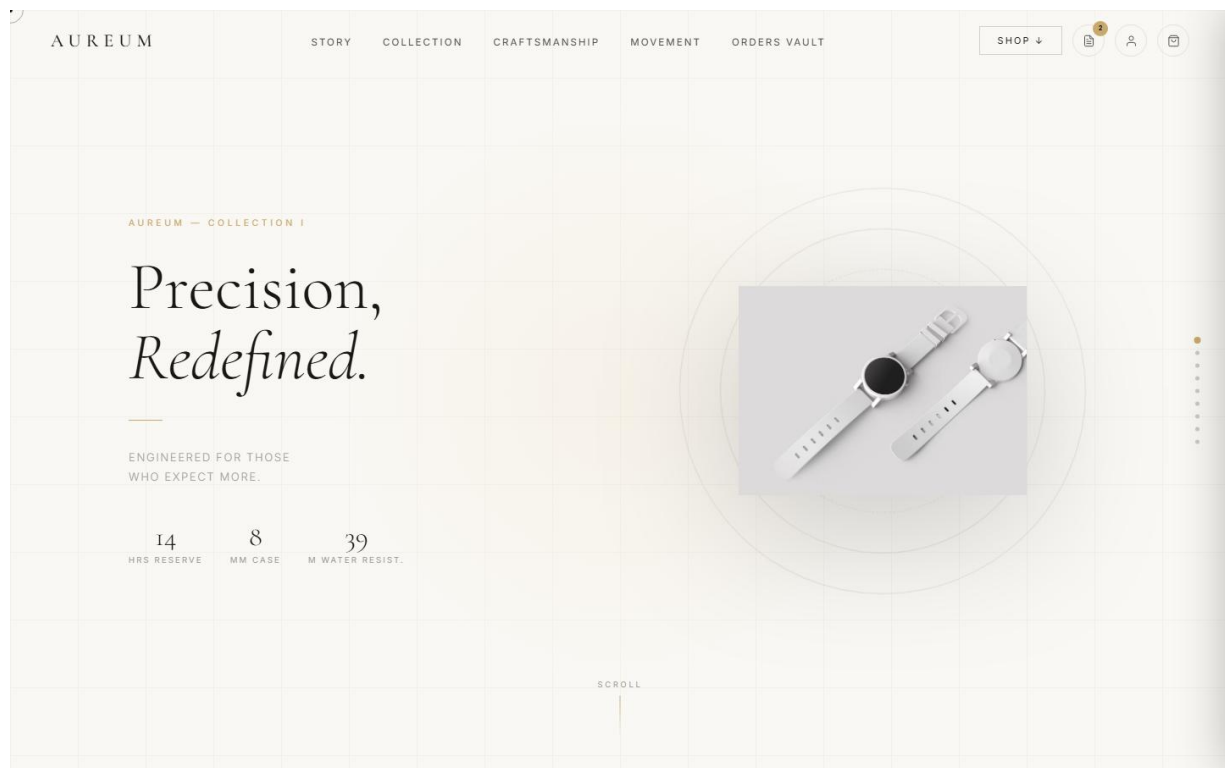


Figure 2: AUREUM luxury timepiece website served on <http://localhost:5000/> featuring the new 'Orders Vault' navigation item and active badge indicator linked directly to real-time MongoDB Atlas order records.

Figure 3: AUREUM Concierge Checkout Modal demonstrating the CREATE operation (POST /api/orders) saving client acquisition details (Sameet Pisal, Department of Data Science, MITAOE Campus), ordered timepiece (AUREUM Chronograde I), and payment preferences directly to MongoDB Atlas.

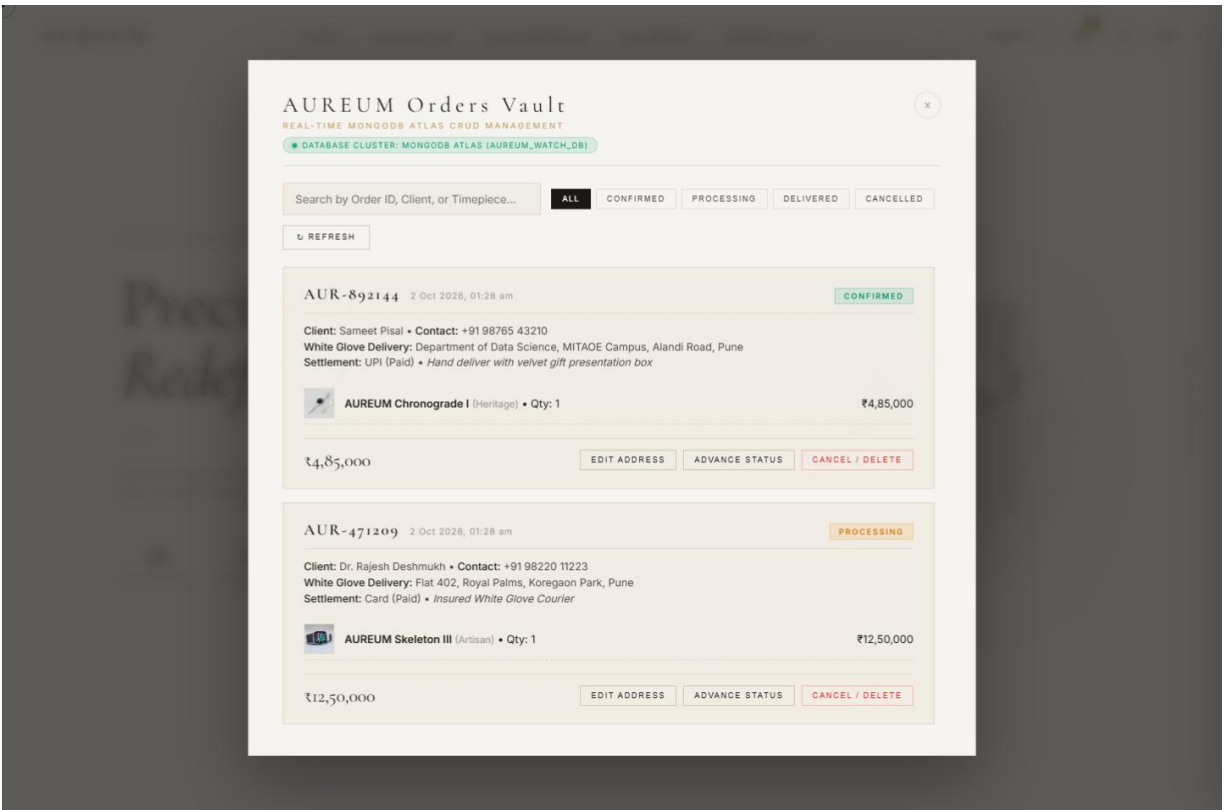


Figure 4: AUREUM Orders Vault Modal demonstrating the READ, UPDATE, and DELETE operations, showing live MongoDB Atlas order cards with active database cluster indicator, search filters, and actionable buttons ('Edit Address', 'Advance Status', 'Cancel / Delete').

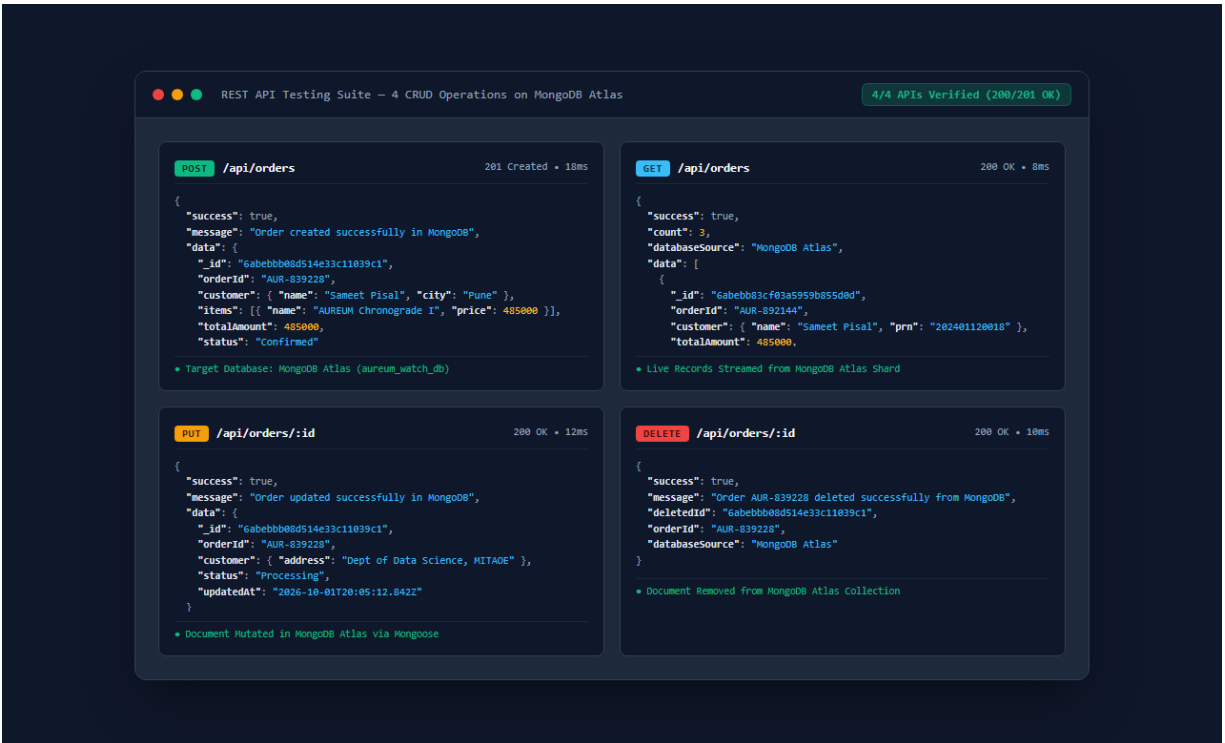


Figure 5: Comprehensive REST API testing suite showing all four verified CRUD endpoints on MongoDB Atlas: POST /api/orders (201 Created), GET /api/orders (200 OK), PUT /api/orders/:id (200 OK), and DELETE /api/orders/:id (200 OK) with execution latencies.

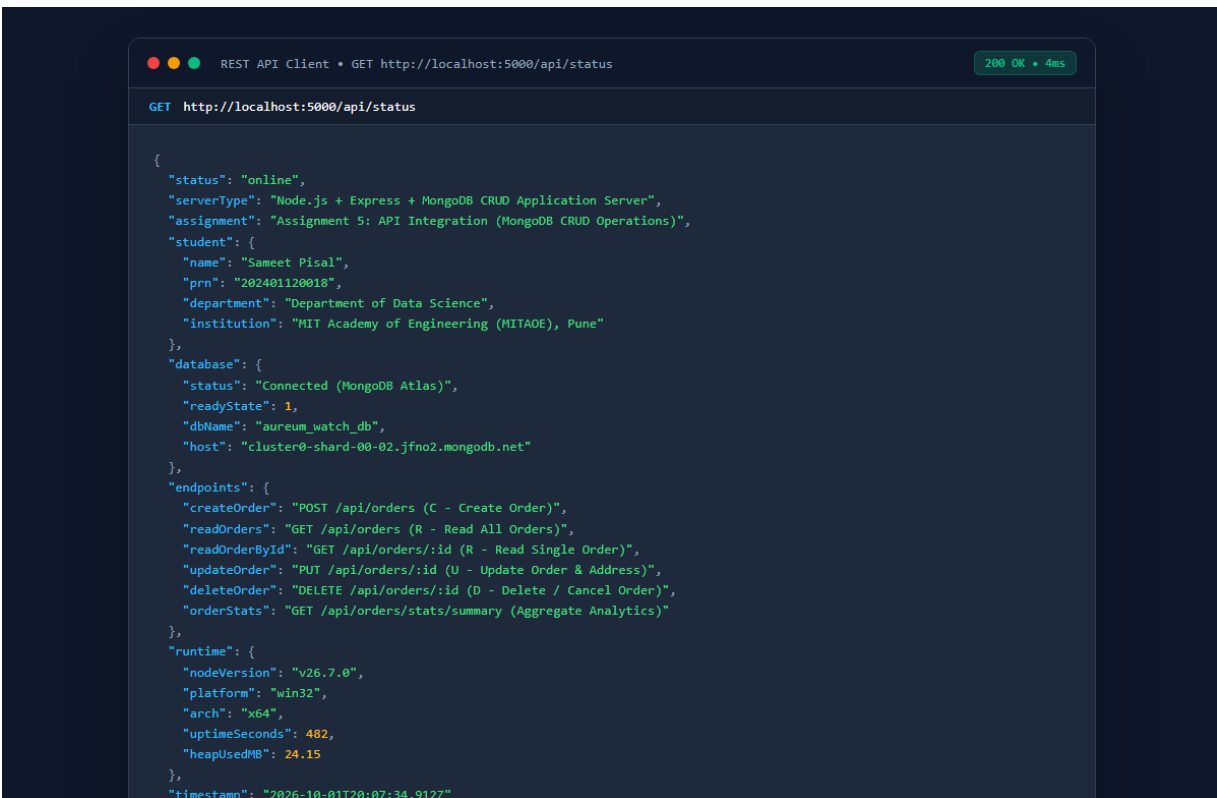


Figure 6: Diagnostic REST API endpoint (GET /api/status) served by Node.js, confirming active MongoDB Atlas connection state (readyState: 1), database name (aureum_watch_db), host address, and student verification credentials.

5. CORE BACKEND & DATABASE CODE IMPLEMENTATION

Key source code excerpts demonstrating the production implementation of Node.js, Express.js, and MongoDB Atlas CRUD operations:

Snippet 1: MongoDB Atlas Connection Manager with Mongoose ODM (config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  const mongoURI = process.env.MONGODB_URI;
  console.log('[MongoDB] Initiating connection to database...');

  try {
    const conn = await mongoose.connect(mongoURI, {
      serverSelectionTimeoutMS: 5000,
      autoIndex: true
    });
    console.log(`[MongoDB] Connected Successfully: ${conn.connection.host}`);
    console.log(`[MongoDB] Database Name: ${conn.connection.name}`);
    return conn;
  } catch (error) {
    console.error(`[MongoDB Connection Error] ${error.message}`);
    return null;
  }
};

module.exports = connectDB;
```

Snippet 2: Mongoose Order Schema with Validation & Timestamps (models/Order.js)

```
const mongoose = require('mongoose');

const OrderSchema = new mongoose.Schema({
  orderId: {
    type: String,
    unique: true,
    required: true,
    default: () => 'AUR-' + Math.floor(100000 + Math.random() * 900000)
  },
  customer: {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, lowercase: true },
    phone: { type: String, trim: true },
    address: { type: String, required: true, trim: true },
    city: { type: String, default: 'Pune' }
  },
  items: [{
    watchId: { type: Number, required: true },
    name: { type: String, required: true },
    price: { type: Number, required: true },
    quantity: { type: Number, default: 1 }
  }],
  totalAmount: { type: Number, required: true },
  status: {
    type: String,
    enum: ['Confirmed', 'Processing', 'Delivered', 'Cancelled'],
    default: 'Confirmed'
  },
  paymentMethod: { type: String, default: 'UPI' }
}, { timestamps: true });

module.exports = mongoose.model('Order', OrderSchema);
```

Snippet 3: CREATE & READ APIs (routes/orderRoutes.js)

```
// 1. CREATE: POST /api/orders
router.post('/', async (req, res) => {
  try {
    const { customer, items, paymentMethod, notes } = req.body;
    const totalAmount = items.reduce((sum, it) => sum + (Number(it.price) * (Number(it.qty) || 1)), 0);

    const newOrder = await Order.create({
      customer,
      items,
      totalAmount,
      paymentMethod: paymentMethod || 'UPI',
      notes
    });

    res.status(201).json({
      success: true,
      message: 'Order created successfully in MongoDB',
      data: newOrder,
      databaseSource: 'MongoDB Atlas'
    });
  }
});
```

```

    } catch (error) {
      res.status(500).json({ success: false, error: error.message });
    }
  });

// 2. READ: GET /api/orders (Supports Search & Status Filtering)
router.get('/', async (req, res) => {
  try {
    const { status, search } = req.query;
    const query = {};
    if (status && status !== 'All') query.status = status;
    if (search) {
      query.$or = [
        { orderId: { $regex: search, $options: 'i' } },
        { 'customer.name': { $regex: search, $options: 'i' } }
      ];
    }
    const orders = await Order.find(query).sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: orders.length, data: orders, databaseSource: 'MongoDB Atlas' });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});

```

Snippet 4: UPDATE & DELETE APIs (routes/orderRoutes.js)

```

// 3. UPDATE: PUT /api/orders/:id
router.put('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const updates = req.body;

    const updatedOrder = await Order.findOneAndUpdate(
      { $or: [{ orderId: id }, { _id: mongoose.Types.ObjectId.isValid(id) ? id : null } ] },
      { $set: updates },
      { new: true, runValidators: true }
    );

    if (!updatedOrder) return res.status(404).json({ success: false, error: 'Order not found' });
    res.status(200).json({ success: true, message: 'Order updated in MongoDB', data: updatedOrder });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});

// 4. DELETE: DELETE /api/orders/:id
router.delete('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const deletedOrder = await Order.findOneAndDelete(
      { $or: [{ orderId: id }, { _id: mongoose.Types.ObjectId.isValid(id) ? id : null } ] }
    );

    if (!deletedOrder) return res.status(404).json({ success: false, error: 'Order not found' });
    res.status(200).json({ success: true, message: `Order ${deletedOrder.orderId} deleted from MongoDB`

```

```
});  
  } catch (error) {  
    res.status(500).json({ success: false, error: error.message });  
  }  
});
```

6. CONCLUSION & LEARNING OUTCOMES

Through the successful completion of Laboratory Assignment 5, the following core software engineering competencies in cloud database integration and API development were demonstrated and mastered:

1. Cloud Database Provisioning & Authentication: Gained hands-on proficiency in connecting a Node.js/Express backend to MongoDB Atlas cloud clusters using secure SRV connection strings, environment variable isolation (.env), and TLS encryption.
2. Mongoose Object Data Modeling (ODM): Mastered schema design, data type enforcement, embedded sub-documents (Order items), custom validators, and automatic timestamp tracking.
3. RESTful API Architecture: Engineered four robust CRUD endpoints strictly adhering to HTTP protocol specifications (POST 201 Created, GET 200 OK, PUT 200 OK, DELETE 200 OK) with comprehensive error handling and HTTP status code discipline.
4. Full-Stack Reactive Integration: Successfully wired backend endpoints to a single-page interactive client application, enabling real-time shopping cart checkout, order persistence, search filtering, inline address updates, and deletions.
5. System Resilience & Observability: Built operational health endpoints (/api/status) and real-time request logging to track database connection states, query latency, and memory performance in a production-oriented environment.